



# Documentation Technique — Detecteur Pro

Guide complet de mise en œuvre d'une application web de sécurité intelligente,  
entièrement fonctionnelle côté client, sans infrastructure backend obligatoire.

FRONTEND-FIRST

MODULAIRE

EVENT-DRIVEN

# Introduction

**Detecteur Pro** est une application web simulant un système de sécurité intelligent. Elle repose sur une architecture **frontend-first** : toute la logique métier, la persistance des données et l'interface utilisateur sont gérées exclusivement côté client, sans nécessiter de serveur backend.

Conçue pour être déployée et démontrée rapidement, l'application intègre des animations de détection, un historique d'événements persistant, et un formulaire de contact par email. Elle constitue une base solide pour un prototype évolutif.

## Fonctionnalités clés

- Simulation animée de la maison et des détecteurs
- Alertes visuelles et sonores en temps réel
- Historique JSON stocké dans LocalStorage
- Envoi d'emails via EmailJS

# Prérequis Techniques

Avant toute mise en œuvre, vérifiez que votre environnement remplit les conditions suivantes.

1

## Navigateur moderne

Chrome, Firefox, Edge ou Safari compatible **ES6+** et **LocalStorage**.  
Aucun plugin ou extension supplémentaire requis.

2

## Connexion Internet

Nécessaire pour l'envoi des emails via **EmailJS**. Le reste de l'application fonctionne entièrement hors ligne.

3

## Éditeur de code

**VS Code**, Sublime Text ou tout IDE supportant HTML, CSS et JavaScript. L'extension Live Server est fortement recommandée.

4

## Serveur local (optionnel)

Pour éviter les restrictions CORS : **Live Server** (VS Code) ou `http-server` via Node.js suffisent.



# Installation et Structure du Projet

## Récupération du projet

Cloner le dépôt GitHub puis ouvrir le dossier racine dans votre éditeur :

```
git clone <url-du-dépôt>  
cd DetecteurPro
```

## Arborescence du projet

```
DetecteurPro/  
├── index.html  
├── style.css  
├── bootstrap.min.css  
├── main.js  
├── modules/  
│   ├── simulation.js  
│   ├── detection.js  
│   ├── historique.js  
│   └── contact.js  
└── assets/
```

# Configuration EmailJS

EmailJS permet l'envoi d'emails directement depuis le navigateur, sans backend. Suivez ces étapes pour l'intégrer correctement.

## Créer compte

Inscrivez-vous sur emailjs.com pour obtenir vos clés.

## Créer service

Ajoutez un service email et créez un template.

## Copier IDs

Récupérez serviceID, templateID et userID.

## Initialiser

Appelez `emailjs.init('VOTRE_USER_ID')` dans `contact.js`.

Une fois les clés récupérées, modifiez `contact.js` en remplaçant `emailjs.init('VOTRE_USER_ID')` par votre identifiant réel. Vérifiez que le formulaire HTML contient bien les champs `name`, `email` et `message` pour que le template fonctionne correctement.

# Architecture Technique

## Principes fondateurs

### Frontend-First

Logique et persistance entièrement côté client. Aucune dépendance serveur obligatoire.

### Modulaire

Chaque fonctionnalité est encapsulée dans un module JS indépendant et réutilisable.

### Event-Driven

Communication inter-modules via `CustomEvent`, garantissant un couplage faible.

## Modules principaux

|                   |                                                            |
|-------------------|------------------------------------------------------------|
| <b>Simulation</b> | Animation de la maison, des détecteurs et des intrus       |
| <b>Détection</b>  | Analyse des zones surveillées et déclenchement des alertes |
| <b>Historique</b> | Sauvegarde et lecture des événements dans LocalStorage     |
| <b>Contact</b>    | Formulaire et envoi d'email via EmailJS                    |

# Stockage des Données — LocalStorage

Detecteur Pro utilise le **LocalStorage** du navigateur pour persister les événements entre les sessions. Les données sont sérialisées en **JSON**, ce qui les rend lisibles et facilement exportables.

Chaque événement enregistré contient : un identifiant unique, le type d'événement, la zone concernée, un horodatage ISO 8601 et le statut de l'alerte. Cette structure facilite l'extension future vers une API REST distante.

## Format de stockage JSON

```
{
  "events": [
    {
      "id": "evt_001",
      "type": "intrusion",
      "zone": "Zone Nord",
      "timestamp":
      "2024-01-15T14:32:00Z",
      "status": "alerte"
    }
  ]
}
```

# Développement et Évolutivité

## Ajouter un module

Créer un fichier JS dans `modules/`, définir un objet ou une classe, puis émettre et écouter des `CustomEvent` pour interagir avec les autres modules.

## Intégration Backend

Pour la synchronisation multi-utilisateurs : ajouter une API REST (Node.js ou Firebase) et remplacer `LocalStorage` par une base de données distante.

## Améliorations futures

Notifications push temps réel via **WebSockets**, et module d'intelligence artificielle pour la détection prédictive des intrusions.





# Bonnes Pratiques de Mise en Œuvre

Respecter ces recommandations garantit un code maintenable, sécurisé et facilement extensible par d'autres développeurs.



## Uniformisation du code

Utiliser **ESLint** pour appliquer des règles de style cohérentes sur l'ensemble des fichiers JavaScript du projet.



## Validation des entrées

Toujours valider les données saisies par l'utilisateur côté client avant tout traitement ou stockage dans LocalStorage.



## Sécurité des clés EmailJS

Stocker les clés `serviceID`, `templateID` et `userID` dans un fichier `.env` et ne jamais les committer dans le dépôt.



## Tests modulaires

Tester chaque module de façon indépendante avant toute intégration, afin de localiser rapidement les régressions éventuelles.

# Conclusion

DetecteurPro est une application prête à l'emploi, démontrant qu'une architecture **frontend-first modulaire** permet de construire rapidement un prototype fonctionnel et maintenable.

## Déploiement immédiat

Aucun serveur requis. Un navigateur moderne suffit pour une démonstration complète.

## Architecture extensible

L'ajout de modules, d'un backend ou de WebSockets reste simple grâce au découplage event-driven.

## Prêt pour la production

Avec ESLint, validation des entrées et gestion sécurisée des clés, le projet respecte les standards professionnels.

